

DemandIQ: Complete Project Technical Guide & Viva Manual

Department of Computer Science and Engineering — CS 781 B.Tech Project

Team Members: Sumit Saha (34900123049), Srijan Das (34900123045), Ishika Chowdhury (34900123015), Jasmin Sultana (34900124065), Arif Sekh (34900123003)

Project Guides: Prof. Sourav Chatterjee / Prof. Supriyo Banerjee — **Repo:** <https://github.com/arifsekh/DemandIQ>

1. What Does Our Project Actually Do? (Simple Words)

A lot of people overcomplicate dynamic pricing. Our project does **two simple things**:

- Predicts Tomorrow's Demand (AI Layer):** It looks at past sales, today's date, whether it's a weekend, festival dates, and Google search trends, then uses an algorithm (XGBoost) to guess: *"How many units will customers want to buy tomorrow?"*
- Calculates a Safe Price (Rules Layer):** A separate Python script takes that demand number, checks how many units are left in the warehouse, checks the competitor's price, and calculates the best price using simple business math.

The Most Important Rule to Remember for Viva

The AI does NOT decide the price directly. The AI only predicts customer demand. A deterministic math formula decides the price. If you let an AI set prices directly, it could hallucinate and price a 2,000 headphone at 20,000 or 10. Separating prediction from pricing is how real enterprise software works.

2. Curated YouTube Learning Resources (Zero Fluff)

Do not watch long, 10-hour beginner videos. Use this table to search for the exact videos you need to understand each part of the project.

Part	Channel	Exact Search Term on YouTube	What to Focus On
Data	Codebasics	"Pandas Tutorial Codebasics"	How DataFrames work, reading CSVs, and how <code>.shift()</code> creates lag columns.
AI Math	StatQuest with Josh Starmer	"StatQuest XGBoost Regression"	Watch parts 1 & 2. Understand how trees calculate residuals and build on previous mistakes.
AI Metrics	StatQuest with Josh Starmer	"StatQuest R-squared clearly explained"	Understand why our $R^2 = 89.32\%$ means the model explains 89% of demand variance.
AI Metrics	CampusX	"CampusX Regression Metrics MAE RMSE"	Understand why MAE is average error in units, while RMSE penalizes big mistakes heavily.
Backend	Sanjeev Thiagarajan (freeCodeCamp)	"FastAPI Course by Sanjeev Thiagarajan"	Watch only the first 90 minutes. Learn what an endpoint is, what a POST request does, and Pydantic validation.
Backend	Krish Naik	"FastAPI deployment with Machine Learning model"	Watch how he loads a <code>.pkl</code> model file with Joblib and returns a prediction in JSON.
Frontend	Data Professor	"Streamlit Tutorial for Beginners Data Professor"	Learn how <code>st.slider()</code> , <code>st.button()</code> , and <code>requests.post()</code> update web pages in pure Python.

3. File-by-File Code Walkthrough (How It Works)

3.1 1. data/generate_data.py (Data Creation)

- **Why it exists:** Private companies (Flipkart, Amazon) do not share raw sales databases. We wrote this script to simulate 365 days of Indian retail data.
- **Key concept — Censored Demand:**

$$\text{Actual Sales} = \min(\text{True Demand}, \text{Available Inventory})$$

If 50 people want headphones today, but we only have 15 in stock, our sales count is 15. The model trains on `True_Demand_Units` (50), not `Actual_Units_Sold` (15), so it doesn't learn to underestimate demand during stockouts.

- **Indian market factors included:** Festival dates (Diwali, Durga Puja, Independence Day), weekend spikes, and logistics cost changes.

3.2 2. models/train.py (The Machine Learning Model)

- **Why we use XGBoost:** It is the industry standard for tabular data. It builds multiple small decision trees where each tree corrects the error of the previous tree.
- **Why a Chronological Split (No random shuffle):** In normal ML, you shuffle data randomly. In time series, you **never shuffle**. You train on the first 80% of the year (Jan–Oct) and test on the remaining 20% (Nov–Dec). Shuffling would leak future information into the past.
- **Feature Engineering:**
 - `Demand_Lag_1`: How many units were sold yesterday.
 - `Demand_Lag_7`: How many units were sold on this exact day last week.
 - `Rolling_Mean_Demand_7D`: The average sales of the last 7 days (smoothes out random daily noise).
- **Model Save:** Saves the trained model into `models/demand_model.pkl` using `joblib`.

3.3 3. engine/pricing_engine.py (Business Logic & Math)

Once XGBoost outputs a predicted demand (e.g., 45 units), this file calculates the price using 4 strict rules:

1. **Cost Floor Rule:** Total Landed Cost = Base Cost + Logistics Cost. The price can **never** go below Landed Cost $\times$ 1.15 (guaranteeing a 15% minimum profit margin). This is how we handle fuel price increases or shipping cost shocks.
2. **Demand Surge Rule:** If predicted demand is higher than average (35 units), the price increases slightly using a dampened multiplier.
3. **Scarcity Rule:** If inventory drops below 15 units while demand is high, the price rises by another 8% to slow down sales and prevent running out of stock completely.
4. **Safety Boundary Rule:** The final price is capped between -15% and $+20\%$ of the regular 1,999 price so customers don't feel cheated.

3.4 4. api/main.py (FastAPI Backend)

- **What it does:** Runs a web server on port 8000.
- **Lifecycle:** When the server starts, it loads `demand_model.pkl` into memory once.
- **Pydantic Validation:** Checks the incoming data. If someone sends a negative inventory or missing price, it rejects the request with an HTTP 422 error before it can crash our model.
- **Endpoint:** `POST /api/v1/pricing/recommend` accepts product data, runs the XGBoost prediction, feeds it to the pricing engine, and returns a JSON response.

3.5 5. dashboard/app.py (Streamlit Frontend)

- **What it does:** Gives store managers an interactive interface with sliders for inventory, competitor price, and logistics costs.
- **How it talks to the backend:** Every time you move a slider, Streamlit sends a POST request with the new values to `http://127.0.0.1:8000`, gets the new price, and updates the metrics and Plotly charts on screen.

3.6 6. simulation/simulate_stream.py (Live Data Simulator)

- Runs an infinite loop that picks random market events (e.g., rival dropped price, festival surge, flash sale) every 3 seconds and sends them to the API. This proves our system handles continuous, real-time data instead of just running once.

4. Team Member Responsibilities

Member	Role	Primary Files	What You Need to Know for Viva
Srijan Das	Data Scientist / ML	models/train.py	Explain XGBoost, lag variables, why chronological split was used, and explain $R^2 = 89.32\%$.
Arif Sekh	Backend / API Lead	api/main.py	Explain how FastAPI works, why Pydantic is used, and how the model is loaded with Joblib.
Sumit Saha	Data / Simulation	data/ & simulation/	Explain censored demand ($\min(\text{Demand}, \text{Inventory})$) and how the real-time loop works.
Ishika Chowdhury	Frontend Engineer	dashboard/app.py	Explain how Streamlit sliders work, how it calls the FastAPI endpoint, and Plotly charts.
Jasmin Sultana	Business Logic / PM	engine/pricing_engine.py	Explain the 4 pricing rules, the cost floor, the $\pm 20\%$ boundary cap, and project documentation.

5. Viva Defense: Questions & Direct Answers

Q1: Where did you get the data since Flipkart and Amazon don't share sales databases?

Answer: “Sir, private e-commerce companies protect daily sales logs under trade secret policies. Instead of using generic or pre-tuned static datasets, we built a calibrated simulation environment based on real Indian market baselines (Flipkart/Amazon electronics prices, Indian festival calendars, and real logistics costs). We also modeled censored demand so the model understands stockout limits.”

Q2: How does your project handle sudden supply chain shocks (like fuel price spikes)?

Answer: “Sir, through our landed cost floor rule in `engine/pricing_engine.py`. When shipping or fuel costs increase, the `Logistics_Cost_INR` input rises. Our deterministic engine recalculates the minimum safe price:

$$P_{\text{floor}} = (\text{Base Cost} + \text{Logistics Cost}) \times 1.15$$

This ensures the business always maintains a minimum 15% profit margin regardless of external fuel or freight inflation.”

Q3: Why not predict the price directly using Deep Learning (like an LSTM or Neural Network)?

Answer: “Two reasons, sir: First, for structured tabular data, gradient boosted decision trees (XGBoost) match or outperform neural networks while requiring much less compute and running with sub-millisecond latency. Second, directly predicting price with ML is risky because models can make irrational pricing predictions. Predicting demand first, then applying bounded math rules to calculate price, ensures complete business and legal safety.”

Q4: What is the difference between MAE and RMSE in your results?

Answer: “MAE (Mean Absolute Error) is 4.13 units, meaning our forecast is off by roughly 4 headphones on average. RMSE (Root Mean Squared Error) is 5.50 units. RMSE squares errors before averaging them, so it penalizes large forecasting mistakes much more heavily than MAE.”

Q5: Why did you use FastAPI and Streamlit instead of standard Flask and HTML/CSS?

Answer: “FastAPI is significantly faster than Flask because it is built natively on ASGI and asynchronous Python. It also provides automatic data validation via Pydantic and generates interactive Swagger documentation out of the box. Streamlit allows rapid development of reactive dashboards in Python without frontend overhead, allowing us to focus on the core AI pipeline.”